SIGNAL RECOVERY Python Drivers Instruction Manual

1.0 Introduction

This document describes the sample Python code which can be used to communicate with ***SIGNAL RECOVERY*** lock-in amplifiers via their RS232, GPIB, USB or Ethernet interfaces. It is assumed that the reader has a basic understanding of the Python environment and language, and that the code is installed on their computer.

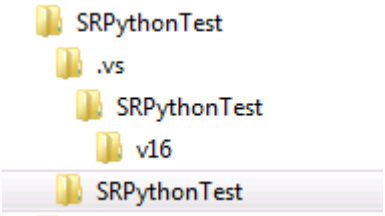
The sample code uses the PyVISA extensions to implement the communications; this in turn needs an underlying installation of VISA, such as NI-VISA, which can be obtained from the NI (National Instruments) website at www.ni.com

2.0 Distribution & Installation

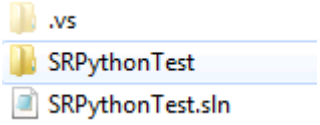
In addition to this document a sample code file `SRPythonTest.py` is provided as well as overall Visual Studio 2019 solution containing it.

The code can be run directly from the command line but the use of a development environment, such as Visual Studio with the optional additional components to support Python programming, makes program development and debugging a great deal easier.

The supplied zip file unpacks to give the following folders:

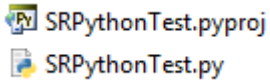


At the top level, the contents are:



`SRPythonTest.sln` is the Visual Studio 2019 solution.

Within the folder SRPythonTest is the sample file `SRPythonTest.py` and the project file:



If using Visual Studio 2019 then simply navidate to and open the `SRPythonTest.sln` solution file. The code was developed and tested using Python version 3.8 and PyVISA version 1.10.1

If using another editor or IDE then use the `SRPythonTest.py` file directly.

3.0 Sample Code Description

This section discusses the function of each code block in turn. After the definition of all the functions (discussed later) the main code block starts with:

```
# main code starts here
import pyvisa
from pyvisa.constants import StopBits, Parity
rm = pyvisa.ResourceManager('C:/Windows/System32/visa32.dll')
```

In order for this to work, PyVISA must be added to to the Python environment. It can be installed using pip:

```
$ pip install -U pyvisa
```

The call to the ResourceManager establishes connection with the NI VISA DLL, in this case `visa32.dll`. This is because it is using a 32 bit version of Python and calling a 32 bit version of NI VISA running on a 32 bit version of Windows. On 64 bit Windows but still using 32 bit Python and 32 bit VISA the line will typically need modifying to:

```
rm = pyvisa.ResourceManager('C:/Windows/sysWOW64/visa32.dll') # 64 bit windows
```

The next section of code simply lists the VISA resources that are present on the machine. The same information, for NI-VISA installations, can be obtained by using the NIMAX program.

```
# Print the list of VISA resources on this computer
rm.list_resources()
print('The VISA resourses present on this computer are: ')
print(rm.list_resources('?*'))
```

There then follow four blocks of code, each of which demonstrates communication via a different interface. These blocks in turn call specific functions to open the selected interface, send a command to the lock-in and return its response, but use a common function to close the interface. Each of these blocks is described in turn in the following sections.

3.1 Ethernet

Ethernet control is supported by the models 7124, 7270, and 7230 (and its variants). The program needs to know the IP address assigned to the lock-in, which can be seen on the front panels of the models 7124 and 7270, or via the `SRInstFinder.exe` utilty, amongst other methods, in the case of the 7230. Please refer to the instructuion manuals for these instruments for more details.

The main block of code for Ethernet control in the sample file is:

```
# Demonstration of Ethernet communications
print('Demonstration of Ethernet communications *************************')
# open the connection with the specified instrument's IP address
inst = Connection_Open_Ethernet('192.168.0.64')
# send a command; returned tuple includes string response, if any, and integer
# status and overload bytes
tReturn = Inst_Query_Command_Ethernet(inst, "VER")
# decode and print the meaning of the status byte
Print_Status_Byte(tReturn[1])
# if response was generated print it
if (tReturn[0] != ''):
    print('Command response: ' + tReturn[0])
# close the connection
Connection_Close(inst)
print('\n')
```

Edit the line:

```
inst = Connection_Open_Ethernet('192.168.0.64')
```

to specify the insttument's IP address. When run, this in turn then calls the function:

```
# opens a connection to the lock-in via Ethernet to the specified IP address.
# Socket 50001 is used
def Connection_Open_Ethernet(sIPAddress):
    print('Open connection via Ethernet')
    inst = rm.open_resource('TCPIP0::' + sIPAddress + '::50001::SOCKET')
    return inst
```

which opens a socket connection to 50001 to the selected IP address via the network interface card 0 (TCPIP0) on the machine. The handle to this connection is returned as `inst`

The core communication routine `Inst_Query_Command_Ethernet` is then called using the handle and the required instrument command, in this case VER which reports the firmware version number. This function is:

```
# sends a write command string sCmd via Ethernet to the lock-in already opened
# as inst and returns the resulting response string and status byte
def Inst_Query_Command_Ethernet(inst, sCmd):
    print('Send query command: ' + sCmd )
    inst.write_raw(sCmd +'\r')
    sResponse = ''
    # read until a CR is received
    sEcho = inst.read_bytes(1).decode('utf8')
    while (sEcho != '\r'):
        sResponse = sResponse + sEcho
        sEcho = inst.read_bytes(1).decode('utf8')
    # now read the status byte
    inst.write_raw('ST\r')
    sStatusByte = ''
    # read until a CR is received
    sEcho = inst.read_bytes(1).decode('utf8')
    while (sEcho != '\r'):
        sStatusByte = sStatusByte + sEcho
        sEcho = inst.read_bytes(1).decode('utf8')
    nStatusByte = int(sStatusByte)
    # mask out bits 4, 5 & 6 which are not consistent across all instruments
    nStatusByte = nStatusByte & 143
    sResponse = sResponse.replace('\n','')
    sResponse = sResponse.replace('\r','')
    # return the response from the instrument and the status byte
    return sResponse, nStatusByte
```

This function writes the command `sCmd` out to the socket, and then reads the response generated by the lock-in character by character until a CR (carriage return) terminator is returned, assembling the characters into `sResponse` string. If the command does not generate a response then the CR terminator is still returned.

The status command ST is then sent and its response returned and converted into an integer, the status byte, of which all but bits 4,5 and 6 is then returned to the calling program. This ensures that critical situations, such as loss of reference lock, or an invalid command parameter, are always notified to the program.

The common function `Print_Status_Byte` is then called, which decodes and prints the

meaning of the five status byte bits which have the same function on all ***SIGNAL RECOVERY*** lock-in amplifiers:

```
def Print_Status_Byte(nStatusByte):
    if (nStatusByte & 1 == 1):
       print('Command Done')
    if (nStatusByte & 2 == 2):
       print('Invalid command')
    if (nStatusByte & 4 == 4):
       print('Command parameter error')
    if (nStatusByte & 8 == 8):
       print('Reference unlock')
    # bits 4, 5 and 6 are instrument model number dependent so are
    # not decoded here
    if (nStatusByte & 128 == 128):
       print('Data Available')
```

Then, if the command returned a response, the response is printed:

```
# if response was generated print it
if (tReturn[0] != ''):
    print('Command response: ' + tReturn[0])
```

before the open `inst` connection is closed by the common function:

```
 # closes the open resource (use for USB, GPIB, RS232, and Ethernet)
def Connection_Close(inst):
    print('Close connection')
    inst.before_close()
    return_status = inst.close()
    return return_status
```

This completes the description of the Ethernet interface code.

3.2 USB

USB control is supported by the models 7124, 7270, and 7230 (and its variants). The program needs to know the instrument's model number since this defines the PID (product identity) and its serial number in order to open communications with the required device. The serial number is shown on a label attached to the rear panel of each unit, and in the case of the models 7124 and 7270 can also be seen on the USB Settings Menu. Please refer to the instructuion manuals for these instruments for more details about to how to access this menu.

The main block of code for USB control in the sample file is:

```
# Demonstration of USB communications
print('Demonstration of USB communications ************************')
# open the connection with the specified instrument model and serial
# number (needed for USB)
inst = Connection_Open_USB('7270', '11063396')
# send a command; returned tuple includes string response, if any, and
# integer status and overload bytes
tReturn = Inst_Query_Command_USB(inst, "VER")
# decode and print the meaning of the status byte
Print_Status_Byte(tReturn[1])
# decode and print the meaning of the overload byte
Print_72XXOverload_Byte(tReturn[2])
# if response was generated print it
if (tReturn[0] != ''):
    print('Command response: ' + tReturn[0])
# close the connection
Connection_Close(inst)
print('\n')
```

Edit the line:

```
inst = Connection_Open_USB('7270', '11063396')
```

to specify the model number and serial number of the connected instrument. When run, this in turn then calls the function:

```
# opens a connection to the lock-in via USB of sModelNumber (string: '7230', '7270',
# or '7124') and sSerialNumber (string)
def Connection_Open_USB(sModelNumber, sSerialNumber):
    print('Open connection via USB')
    if (sModelNumber == '7270'):
        inst = rm.open_resource('USB0::0x0A2D::0x001B::' + sSerialNumber + '::RAW')
    if (sModelNumber == '7230'):
        inst = rm.open_resource('USB0::0x0A2D::0x0027::' + sSerialNumber + '::RAW')
    if (sModelNumber == '7124'):
        inst = rm.open_resource('USB0::0x0A2D::0x0018::' + sSerialNumber + '::RAW')
    return inst
```

which uses the supplied model and serial number to determine the correct VISA resource string to opens a connection to the instrument. The handle to this connection is returned as `inst`

The core communication routine `Inst_Query_Command_USB` is then called using the handle and the required instrument command, in this case VER which reports the firmware version number. This function is:

```
# sends a write command string sCmd via USB to the lock-in already opened as inst
# and returns the resulting response string, status nad overload bytes
def Inst_Query_Command_USB(inst, sCmd):
    print('Send query command: ' + sCmd)
    inst.write_raw(sCmd)
    sResponse = inst.read()
    # read the status and overload bytes
    nStb = bytes((sResponse[len(sResponse)-2:len(sResponse)-1:]),'utf-8')
    nOvb = bytes((sResponse[len(sResponse)-1:len(sResponse):]),'utf-8')
    nStatusByte = int(nStb[0])
    # mask out bits 4, 5 & 6 which are not consistent across all instruments
    nStatusByte = nStatusByte & 143
    nOverloadByte = int(nOvb[0])
    # strip the returned response of the line feed, status & overload bytes, and
    # the null terminator
    sResponse = sResponse[0:len(sResponse)-4:]
    # return the response from the instrument, the status byte, and the overload byte
    return sResponse, nStatusByte, nOverloadByte
```

This function writes the command `sCmd` out to the instrument, and then reads the response generated by the lock-in into `sResponse` string.

For USB communications the instrument terminates the response with the status and overload bytes (rather than needing to send a further command to determine these), so these are extracted. The status byte, except bits 4,5 and 6 is then returned to the calling program together with the overrload byte and the response.

The common function `Print_Status_Byte` is then called, which decodes and prints the meaning of the five status byte bits which have the same function on all ***SIGNAL RECOVERY*** lock-in amplifiers, as well as `Print_72XXOverload_Byte` which decodes and prints the meaning of this byte on 72XX series instruments:

```
def Print_72XXOverload_Byte(nOverloadByte):
    if (nOverloadByte & 1 == 1):
       print('X(1) output overload')
    if (nOverloadByte & 2 == 2):
       print('Y(1) output overload')
    if (nOverloadByte & 4 == 4):
       print('X2 output overload')
    if (nOverloadByte & 8 == 8):
       print('Y2 output overload')
    if (nOverloadByte & 16 == 16):
       print('CH1 output overload')
    if (nOverloadByte & 32 == 32):
       print('CH2 output overload')
    if (nOverloadByte & 64 == 64):
       print('CH3 output overload')
    if (nOverloadByte & 128 == 128):
       print('CH4 output overload')
```

Then, if the command returned a response, the response is printed:

```
# if response was generated print it
if (tReturn[0] != ''):
    print('Command response: ' + tReturn[0])
```

before the open `inst` connection is closed by the common function:

```
# closes the open resource (use for USB, GPIB, RS232, and Ethernet)
def Connection_Close(inst):
    print('Close connection')
    inst.before_close()
    return_status = inst.close()
    return return_status
```

This completes the description of the USB interface code.

3.3 RS232

RS232 control is supported by all ***SIGNAL RECOVERY*** lock-in amplifiers. It generally requires a null modem cable to connect the instrument to the computer, with details of the required wiring between the two connectors being given in each unit's instructuion manual. The default communication settings are 7 data bits, 1 stop bit, even parity for all units, with 19200 baud for the models 7124, 7270, and 7230, and 9600 baud for other units.

The main block of code for RS232 control in the sample file is:

```
# Demonstration of RS232 communications
print('Demonstration of RS232 communications *************************')
# open the connection with the specified port and baud rate (other params are
# set to 7 data bits, 1 stop bit, even parity)
inst = Connection_Open_RS232('ASRL1', "19200")
# send a command; returned tuple includes string response, if any, and
# integer status byte
tReturn = Inst_Query_Command_RS232(inst, "VER")
# decode and print the meaning of the status byte
Print_Status_Byte(tReturn[1])
# if response was generated print it
if (tReturn[0] != ''):
    print('Command response: ' + tReturn[0])
# close the connection
Connection_Close(inst)
print('\n')
```

Edit the line:

```
inst = Connection_Open_RS232('ASRL1', "19200")
```

to specify the serial port (ASRL1 = COM1, ASLR2 = COM2, etc) and baud rate used for the connected instrument. When run, this in turn then calls the function:

```
# opens a connection to the lock-in via RS232 of sPort, sBaudRate (strings)
def Connection_Open_RS232(sPort, sBaudRate):
    print('Open connection via RS232')
    inst = rm.open_resource(sPort)
    inst.baud_rate = int(sBaudRate)
    inst.parity = Parity.even
    inst.data_bits = 7
    return inst
```

which uses the supplied port and baud rate determine the correct VISA resource string to open a connection to the instrument, and set the baud rate, parity, and data bits. The handle to this connection is returned as `inst`

The core communication routine `Inst_Query_Command_RS232` is then called using the

hamdle and the required instrument command, in this case VER which reports the firmware version number.

```
# sends a write command string sCmd via RS232 to the lock-in already opened as inst
# and returns the resulting response string and status byte
def Inst_Query_Command_RS232(inst, sCmd):
    print('Send query command: ' + sCmd)
    # serial port commands need sending one character at a time and checking for
    # handshake
    for i in range(len(sCmd)):
        inst.write_raw(sCmd[i])
        sEcho = inst.read_bytes(1).decode('utf8')
    # write the terminator
    inst.write_raw('\r')
    sResponse = ''
    # read until recieve a prompt of ? or *
    sEcho = inst.read_bytes(1).decode('utf8')
    while (sEcho != '?') and (sEcho != '*'):
        sResponse = sResponse + sEcho
        sEcho = inst.read_bytes(1).decode('utf8')
    sResponse = sResponse.replace('\n','')
    sResponse = sResponse.replace('\r','')
    # set returned status byte to Comamnd Done
    nStatusByte = 1
    if (sEcho == '*'):
       nStatusByte = 1
    if ((sEcho == '?') & (sCmd != 'ST')):
        # send the status command to get instrument status except if this is being
        # called by a ST command itself
        sStatus = Inst_Query_Command_RS232(inst, 'ST')
        nStatusByte = int(sStatus[0])
    # mask out bits 4, 5 & 6 which are not consistent across all instruments
    nStatusByte = nStatusByte & 143
    # return the response from the instrument and the status byte
    return sResponse, nStatusByte
```

The RS232 interface uses character by character handshaking to verify that commands have been correctly received and so cannot use communications routines which involve writing the whole command out to the interface as a single string. Hence the command `sCmd` is sent to the instrument one character at a time and the echoed response read back before the next character is sent. Finally a carriage return terminator is written.

The instrument then returns the response, if any, followed by prompt characters of either * or ? This response is read one character at a time and assembled into the `sResponse` string until the prompt character is reached. If this is a ? then the status command ST is then sent to read the the status byte which, except for bits 4,5 and 6 is then returned to the calling program together with the response.

The common function `Print_Status_Byte` is then called, which decodes and prints the meaning of the five status byte bits which have the same function on all ***SIGNAL RECOVERY*** lock-in amplifiers.

Then, if the command returned a response, the response is printed:

```
# if response was generated print it
if (tReturn[0] != ''):
    print('Command response: ' + tReturn[0])
```

before the open `inst` connection is closed by the common function:

```
  # closes the open resource (use for USB, GPIB, RS232, and Ethernet)
def Connection_Close(inst):
    print('Close connection')
    inst.before_close()
    return_status = inst.close()
    return return_status
```

This completes the description of the RS232 interface code.

3.4 GPIB

GPIB control is supported by older ***SIGNAL RECOVERY*** lock-in amplifiers, including the models 7265, 7225, 7280, 7260, 7220, 5210, and 5209. It needs a GPIB interface for the computer, typically an NI GPIB-USB or a PCI plug in card and connecting GPIB cable. Each instrument connected to the same bus must have a unique GPIB address, for which the default on ***SIGNAL RECOVERY*** instruments is 12. Please refer to the instructuion manuals for these instruments for more details on how to adjust this setting.

The main block of code for GPIB control in the sample file is:

```
# Demonstration of GPIB communications
print('Demonstration of GPIB communications **************************')
# open the connection with the specified instrument GPIB address
inst = Connection_Open_GPIB('12')
# send a command; returned tuple includes string response, if any, and
# integer status byte
tReturn = Inst_Query_Command_GPIB(inst, "VER")
# decode and print the meaning of the status byte
Print_Status_Byte(tReturn[1])
# if response was generated print it
if (tReturn[0] != ''):
    print('Command response: ' + tReturn[0])
# close the connection
Connection_Close(inst)
print('\n')
```

Edit the line:

```
inst = Connection_Open_GPIB('12')
```

to specify the GPIB address if it is other than 12. When run, this in turn then calls the function:

```
  # opens a connection to the lock-in via GPIB of sGBIBAddress (string)
def Connection_Open_GPIB(sGBIBAddress):
    print('Open connection via GPIB')
    inst = rm.open_resource('GPIB0::' + sGBIBAddress + '::INSTR')
    return inst
```

which uses the supplied port and baud rate determine the correct VISA resource string to open a connection to the instrument using the GPIB interface GPIB0. The handle to this connection is returned as `inst`

The core communication routine `Inst_Query_Command_GPIB` is then called using the handle and the required instrument command, in this case VER which reports the firmware version number.

```
# sends a write command string sCmd via GPIB to the lock-in already opened as inst
# and returns the resulting response string and status byte
def Inst_Query_Command_GPIB(inst, sCmd):
    print('Send query command: ' + sCmd)
    inst.write(sCmd)
    sResponse =  ''
    # repeatedly read status byte until bit 7 is asserted = data available or
    # until bit 1 = command done is asserted
    nStatusByte = int(inst.stb)
    while (nStatusByte & 0x80 != 0x80):
        nStatusByte = int(inst.stb)
        if (nStatusByte & 0x01 == 0x01):
            break
    if (nStatusByte & 0x80 == 0x80):
        # data available, read it back
        sResponse = inst.read()
        # strip the CR LF terminator
        sResponse = sResponse[0:len(sResponse)-2:]
    # read status byte until bit 1 is asserted = command done
    while (nStatusByte & 0x01 != 0x01):
        nStatusByte = int(inst.stb)
    # mask out bits 4, 5 & 6 which are not consistent across all instruments
    nStatusByte = nStatusByte & 143
    # return the response from the instrument and the status byte
    return sResponse, nStatusByte
```

The `sCmd` command is written to the instrument. Repeated serial polls are then performed until bit 7 or bit 1 are asserted. If bit 7 is asserted, which indicates there is a response, a read is performed and any termination characters removed; if bit 1 is asserted, meaning "command done" then this stage is bypassed.

Finally further serial polls are performed to verify the command is complete before the response and the status byte, except for bits 4,5 and 6, are then returned to the calling program.

The common function `Print_Status_Byte` is then called, which decodes and prints the meaning of the five status byte bits which have the same function on all ***SIGNAL RECOVERY*** lock-in amplifiers.

Then, if the command returned a response, the response is printed:

```
# if response was generated print it
if (tReturn[0] != ''):
    print('Command response: ' + tReturn[0])
```

before the open `inst` connection is closed by the common function:

```
# closes the open resource (use for USB, GPIB, RS232, and Ethernet)
def Connection_Close(inst):
    print('Close connection')
    inst.before_close()
    return_status = inst.close()
    return return_status
```

This completes the description of the GPIB interface code.

4.0 Operation

When the computer is connected to a model 7270 via all three RS232, USB, and Ethernet interfaces and to a model 7265 via the GPIB interface and the program is run, the resulting output is:

```
The VISA resources present on this computer are:
('USB0::0x0A2D::0x001B::11063396::RAW',
'TCPIP0::192.168.0.64::50001::SOCKET', 'ASRL1::INSTR',
'ASRL6::INSTR', 'ASRL10::INSTR', 'GPIB0::INTFC',
'GPIB0::12::INSTR')
Demonstration of Ethernet communications
*************************
Open connection via Ethernet
Send query command: VER
Command Done
Command response: 2.22
Close connection

Demonstration of USB communications *************************
Open connection via USB
Send query command: VER
Command Done
Command response: 2.22
Close connection

Demonstration of RS232 communications *************************
Open connection via RS232
Send query command: VER
Command Done
Command response: 2.22
Close connection

Demonstration of GPIB communications *************************
Open connection via GPIB
Send query command: VER
Command Done
Command response: 10.5
Close connection
```

5.0 Using the Sample Code

It is recommended that a decision be made as to which of the possible interfaces will be used and then just the sections relating to this interface be copied to a new file. Check that the new code works correctly with the target instrument before then adding further functionality.

The supplied routines cover the vast majority of possible applications. However, some instruments have the ability to return multiple lines of data in response to a single command (e.g. when downloading data from the curve buffer), or to return data in binary rather than ASCII format. If it is intended to support these then it is suggested that further functions be defined just to handle them, rather than attempting to create a universal function that can handle everything. This will make debugging programs much easier.

6.0 Further Advice

If you require further help or assistance then please contact us via email at **si.info@ametek.com**

SIGNAL RECOVERY

SIGNAL RECOVERY is part of AMETEK Advanced Measurement Technology, Inc

801 SOUTH ILLINOIS AVENUE
OAK RIDGE
TN 37831-2011
USA
Phone: +1 865 482 4411
Fax: +1 865 481 2410

SPECTRUM HOUSE,
1 MILLARS BUSINESS CENTRE, FISHPONDS CLOSE
WOKINGHAM, BERKS RG41 2TZ
UNITED KINGDOM
Phone: +44 (0) 1252 556800
Fax: +44 (0) 1252 556899

E-mail: si.info@ametek.com **Web Site:** www.signalrecovery.com